MCP_ARCHITECTURE

Helios — MCP Architecture & Implementation Spec

Companion to: `HELIOS_PROJECT_BIBLE.md` §18 · **Version:** v1.0 · **Date:** 2026-06-03 **Specifies artifacts:** `A6.0` (shared scaffolding), `A6.1` warehouse-mcp, `A6.2` semantic-mcp, `A6.3` stats-mcp, `A6.4` experiment-mcp, `A6.5` report-mcp core, `A7.4` report-mcp memory tools · **Build milestone:** M6 / M6b (DEPENDENCY_MAP §3).

Purpose. Bible §18 is the architecture overview; this doc is the build-grade contract an engineer implements against. It pins every tool's typed I/O, the error taxonomy, the guardrail state machine, transport/auth/config, the authoritative per-agent allow-list, the registry binding, the testing strategy, and reference skeletons. Where this doc and Bible §18 prose differ on the two items below, **this doc is authoritative** (and the Bible should be reconciled to it):

> **Reconciliation 1 — per-agent allow-list.** Bible §18.9 (simplified) and §19.1 (fuller) disagree. The authoritative list is §10 below: the union each agent actually needs in the §19.6 run sequence, with the rule that *any agent holding* `run_query` *also holds* `dry_run` (rule G3). **Reconciliation 2 — report-mcp memory backend.** Bible §18.10 loosely says `postgres://helios/memory`; §22 specifies BigQuery `helios_memory` + a vector store. The authoritative backend is §22's: BigQuery system-of-record + vector store for ANN recall.

---

1. Why MCP (the boundary, in one paragraph)

The product thesis is **grounding over generation**: the LLM must never author raw SQL and never compute a statistic in token-space. MCP makes that boundary *physically enforceable* rather than a prompt-time suggestion — the model is given tools, not a database handle or a Python REPL. Three principles map to three design rules: **grounding** → `semantic-mcp` is the only path to SQL; **determinism** → `stats-mcp` is the only path to math; **least privilege** → only `warehouse-mcp` holds a BigQuery client, and each agent sees only the tools in its allow-list (§10). A buggy or compromised server's blast radius is bounded by its tool catalog. (Full rationale: Bible §18.1–18.2.)

---

2. The guardrail chain (load-bearing invariants)

Every governed datapoint flows through this fixed sequence. The chain is enforced by tool preconditions, not by agent goodwill.

```
agent picks canonical metric/dim names
      │
      ▼  (G1) ONLY path to SQL
semantic-mcp.build_query(metric, dims, filters, window) ──► governed SQL string
      │
      ▼  (G3) cost gate BEFORE execution
warehouse-mcp.dry_run(sql) ──► {bytes, valid}     ── over budget? → reject; agent narrows scope
      │
      ▼  run_query refuses SQL not dry-run'd this run (sha256 gate) AND caps max_bytes_billed
warehouse-mcp.run_query(sql, max_bytes_billed) ──► rows (aggregated, never row-level dumps to LLM)
      │
      ▼  (G2) ONLY path to math          (G4) reconcile aggregates ≤0.5% drift
stats-mcp.* (decompose/significance/forecast)  +  warehouse-mcp.reconcile(metric, grain)
      │
      ▼  adversarial gate
Critic refutes (confound / sample / seasonality / data-quality) ──► PASS | DOWNGRADE | DROP
      │
      ▼  only PASS findings persist
report-mcp.render_brief( … ) → report-mcp.save_diagnosis( … )   (writes memory, §22)
```

Invariants restated as preconditions: **(I1)** `run_query(sql)` raises `NotDryRunFirst` unless `normalize_hash(sql)` was seen by a `dry_run` in the same run. **(I2)** `run_query` caps `maximum_bytes_billed = min(arg, byte_budget)`; over-scan → `ByteBudgetExceeded`. **(I3)** `build_query` raises on any name not in the registry (`UnknownMetric` / `UnknownDimension`) — SQL can only reference real GA4 columns. **(I4)** numbers in a brief are `stats-mcp` / `experiment-mcp` outputs verbatim — the LLM never arithmetic.

---

3. Topology, transport, auth, configuration

Concern	Decision
Transport	`semantic`, `stats`, `experiment`, `report` run over **stdio** (subprocess, JSON-RPC on stdin/stdout — lowest latency, no network surface). `warehouse-mcp` runs over **streamable HTTP** so it lives in a network boundary with the BigQuery service account and is shared across runs.
Auth	`warehouse-mcp`: GCP service account via ADC (`GOOGLE_APPLICATION_CREDENTIALS`), scoped to `roles/bigquery.dataViewer` + `roles/bigquery.jobUser`, **read-only** on `bigquery-public-data`; HTTP endpoint guarded by bearer token `HELIOS_WH_TOKEN`. stdio servers inherit **no** warehouse credentials. `report-mcp`: write access to the `helios_memory` dataset + vector store only.
Statelessness	All tools are stateless request/response **except** `report-mcp`'s memory tools (`save_diagnosis` / `recall_prior`), an explicit durable side effect backed by BigQuery + a vector store — not in-process session state. Statelessness makes runs idempotent and horizontally scalable.
Registration	The Claude Agent SDK (MCP client) loads `mcp_servers.yaml`, spawns/connects each server, fetches its tool manifest, and exposes the **per-agent-filtered** union of tools (§10).
Config	Declarative `mcp_servers.yaml` (below). Byte budget = **5 GiB** (`5368709120`); stats RNG seed = **1729**; registry path = the reconciled canonical `./models/semantic/semantic_layer.yaml`.

```yaml
# mcp_servers.yaml  (repo root)
servers:
  warehouse-mcp:
    transport: http
    url: https://helios-wh.internal:8443/mcp
    auth: {type: bearer, token_env: HELIOS_WH_TOKEN}
    config:
      dataset: bigquery-public-data.ga4_obfuscated_sample_ecommerce
      marts_project: helios
      byte_budget: 5368709120        # 5 GiB hard cap per run
      require_dry_run: true
  semantic-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.semantic"]
    config: {registry: ./models/semantic/semantic_layer.yaml}   # the A5.1 keystone
  stats-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.stats"]
    config: {rng_seed: 1729}
  experiment-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.experiment"]
  report-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.report"]
    config:
      memory_dataset: helios_memory                # BigQuery system of record (§22)
      vector_store: ${HELIOS_VECTOR_STORE_URI}     # ANN recall for recall_prior
```

Required environment variables

`GOOGLE_APPLICATION_CREDENTIALS` · `HELIOS_WH_TOKEN` · `HELIOS_VECTOR_STORE_URI` · (`ANTHROPIC_API_KEY` is the agent layer, not MCP).

4. Repository layout (`helios/mcp/`)

```
helios/mcp/
├─ __init__.py
├─ base.py          # A6.0 shared FastMCP scaffolding: server factory, error types,
│                   #       JSON-RPC framing, tool-call audit wrapper, config loader
├─ schemas.py       # shared pydantic models / JSON Schemas for every tool's I/O
├─ warehouse.py     # A6.1  (streamable-http; sole BigQuery client; dry_run/run_query gate)
├─ semantic.py      # A6.2  (stdio; loads the A5.1 registry; build_query resolver)
├─ stats.py         # A6.3  (stdio; seeded; decompose_change + significance + forecast + cohort + rfm)
├─ experiment.py    # A6.4  (stdio; power/runtime/design; no data access)
└─ report.py        # A6.5 core (render_brief/export) + A7.4 memory tools (save_diagnosis/recall_prior)
mcp_servers.yaml    # registration/config (repo root)
```

5. Error taxonomy (shared, mapped to JSON-RPC error codes)

All tools raise from this closed set; the client surfaces `code` + `message` + `data` to the agent so it can self-correct (e.g. narrow scope on `ByteBudgetExceeded`, re-plan on `UnknownDimension`).

Error	JSON-RPC code	Raised by	Agent recovery
`UnknownMetric`	-32001	semantic, warehouse.reconcile	re-plan against `get_metric`/registry; never fall back to free SQL (G5)
`UnknownDimension`	-32002	semantic	re-plan against `list_dimensions()`
`DimensionNotPermitted`	-32003	semantic.build_query	drop the dim or pick a metric that whitelists it
`InvalidFilter` / `InvalidWindow`	-32004	semantic.build_query	fix predicate/window shape
`SqlSyntaxError` / `SchemaError`	-32010	warehouse.dry_run	should never happen (governed SQL) → hard stop + audit
`NotDryRunFirst`	-32011	warehouse.run_query	call `dry_run` first (G3)
`ByteBudgetExceeded`	-32012	warehouse.run_query	narrow window/dims, re-dry_run
`QueryTimeout`	-32013	warehouse.run_query	retry w/ backoff (≤3), then prune branch
`AuthError` / `DatasetNotFound` / `TableNotFound`	-32020	warehouse	abort run, audit
`InsufficientData`	-32030	stats	widen window or prune the branch
`SegmentMismatch`	-32031	stats.decompose_change	align t0/t1 segment sets
`ZeroSample` / `ZeroTraffic` / `InvalidRate`	-32032	stats, experiment	guard inputs
`EmptyFindings`	-32040	report.render_brief	emit "no material change" brief
`ValidationError`	-32041	report.save_diagnosis	fix finding envelope shape
`UnsupportedFormat`	-32042	report.export	pick md/pdf/slack

6. Server specifications

Notation: inputs are typed; `?` = optional with default; outputs are JSON object shapes.

6.1 `warehouse-mcp` — governed execution (enforces verify-then-trust)

Sole holder of a BigQuery client. Executes only SQL that already passed `semantic-mcp`. HTTP transport.

Tool	Signature	Output	Errors
`list_tables`	`(dataset?: str)`	`{tables: [{name,row_count,size_bytes,shard_min,shard_max}]}`	`DatasetNotFound`, `AuthError`
`describe_table`	`(table: str)`	`{name, schema:[{name,type,mode}], num_rows}`	`TableNotFound`
`dry_run`	`(sql: str)`	`{valid:bool, total_bytes_processed:int, estimated_cost_usd:float, referenced_tables:[str]}`	`SqlSyntaxError`, `SchemaError`
`run_query`	`(sql: str, max_bytes_billed: int)`	`{rows:[obj], row_count:int, bytes_processed:int, job_id:str}`	`NotDryRunFirst`, `ByteBudgetExceeded`, `QueryTimed`
`reconcile`	`(metric: str, grain: str)`	`{metric, grain, canonical_total:float, source:"warehouse"}`	`UnknownMetric`

```
// dry_run → response
{"valid" true "total_bytes_processed" 248901376 "estimated_cost_usd" 0.00155 "referenced_tables" ["events_2021*"]}
```

Invariants: I1 (sha256 dry-run gate), I2 (byte cap). `reconcile` recomputes a metric's canonical total directly from the governed mart so fact-derived results can be checked to ≤0.5% (rule G4).

6.2 `semantic-mcp` — the ONLY path to SQL (enforces grounding)

Loads the A5.1 registry (`models/semantic/semantic_layer.yaml`) at startup, compiles it (referential-integrity check, §8), and serves governed SQL. stdio. **Cannot execute SQL.**

Tool	Signature	Output	Errors
`get_metric`	`(name: str)`	`{name, type, grain, agg?, sql?, numerator?, denominator?, expr?, depends_on: [str], version}`	`UnknownMetric`
`list_dimensions`	`()`	`{dimensions: [{name, type, sql_expr, entity}]}`	—
`build_query`	`(metric: str|[str], dims: [str], filters: [{dim,op,value}], window: {start,end}|str)`	`{sql:str, governed:true, metrics:[str], dims:[str]}`	`UnknownMetric` , `UnknownDimension` , `DimensionNotPermitted` , `InvalidFilter` , `InvalidWinc`

```
// build_query → response (abbrev.)
{"governed" true "metrics" ["sessions","purchasing_sessions","session_conversion_rate"] "dims" ["device_category"]
 "sql" "WITH s AS (SELECT device_category, session_key, MAX(reached_purchase) AS purchased FROM `helios.marts.fct_funnel` WHERE event_date BETWEEN
        '2021-01-01' AND '2021-01-31' GROUP BY 1,2) SELECT device_category, COUNT(DISTINCT session_key) AS sessions, COUNTIF(purchased) AS
        purchasing_sessions, SAFE_DIVIDE(COUNTIF(purchased),COUNT(DISTINCT session_key)) AS session_conversion_rate FROM s GROUP BY 1"}
```

Multi-metric rule: all metrics in one `build_query` must share a `grain` (else `DimensionNotPermitted` /grain error); `count` / `sum` / `ratio` / `derived` composed in a single statement per the resolver (§9). **Anti-hallucination:** the model passes only names; physical columns live exclusively in registry `sql` fields owned by analytics engineers.

6.3 `stats-mcp` — the ONLY path to math (enforces determinism)

Seeded (`rng_seed: 1729`); scipy/statsmodels/prophet. The model passes arrays in and gets numbers out. stdio. No data access.

Tool	Signature	Output	Errors
`detect_anomaly`	`(series:[{t,value}], method:"stl"|"ewma"|"iqr")`	`{anomalies: [{t,value,score,direction}]}`	`InsufficientData`
`decompose_change`	`(metric:str, dim:str, t0: [{seg,w,r}], t1:[{seg,w,r}])`	`{delta_R, mix_effect, rate_effect, interaction, by_segment: [{seg,mix,rate,interaction}]}`	`SegmentMismatch` , `InsufficientData`
`significance_test`	`(a:{n,x}, b:{n,x}, kind:"proportion"|"mean")`	`{p_value, effect_size, ci_low, ci_high, significant:bool}`	`ZeroSample`
`forecast`	`(series:[{t,value}], horizon:int)`	`{forecast:[{t,yhat,lo,hi}], model:str}`	`InsufficientData`
`cohort_retention`	`(events, cohort_grain, periods)`	`{matrix:[[float]]}`	`InsufficientData`

Tool	Signature	Output	Errors
`rfm_segment`	`(users: [{id,recency,frequency,monetary}])`	`{segments:[{id,r,f,m,label}]}`	—

`decompose_change` implements the FOUNDATION identity exactly: `mix=Σ Δw·r(t0)`, `rate=Σ w(t0)·Δr`, `interaction=Σ Δw·Δr`, `ΔR=mix+rate+interaction`. (Golden test in §11.)

6.4 `experiment-mcp` — statistically-defensible backlog

Turns findings into powered test cards. stdio. No data access (consumes baselines from prior tools).

Tool	Signature	Output	Errors
`power_analysis`	`(baseline:float, mde:float, alpha?:float=0.05, power?:float=0.8)`	`{n_per_arm:int, total_n:int}`	`InvalidRate`
`runtime_estimate`	`(n:int, traffic:float)`	`{days:float, weeks:float}`	`ZeroTraffic`
`design_experiment`	`(hypothesis:str, metric:str)`	`{test_card:{hypothesis, primary_metric, mde, n_per_arm, runtime_days, guardrail_metrics}}`	`UnknownMetric`

`design_experiment` validates `metric` (and any guardrail metric) against the semantic registry, so a test card can only target governed metrics (e.g. `net_revenue`, not `refund_rate`).

6.5 `report-mcp` — narration (`A6.5` core) + memory (`A7.4` tools)

Renders the Decision Brief and is the **only** memory-I/O path for agents. The render/export tools are stateless (T6); the memory tools depend on the BigQuery `helios_memory` store + vector store (T7).

Tool	Tier	Signature	Output	Errors	Side effects
`render_brief`	A6.5	`(findings:[obj])`	`{brief_md:str, brief_html:str}`	`EmptyFindings`	none
`export`	A6.5	`(format:"pdf"|"slack"|"md")`	`{uri:str}`	`UnsupportedFormat`	writes artifact
`save_diagnosis`	A7.4	`(diagnosis:obj)`	`{id:str, saved:true}`	`ValidationError`	**writes `helios_memory` + upserts vector store**
`recall_prior`	A7.4	`(metric:str, segment:str)`	`{prior: [{id,t,summary,action_status}]}`	—	reads `helios_memory` + ANN vector search

`save_diagnosis` writes a `diagnosis_history` row (§22.1) and upserts `(finding_id, embedding, metric, root_cause_label)` to the vector store. `recall_prior` runs the hybrid query (exact BigQuery filter on `metric` + `dimension_slice` ∪ vector ANN on the candidate's summary), decayed by `exp(-age_days/60)`.

---

7. semantic-mcp ↔ registry binding (the anti-hallucination core)

1. **Load.** On startup, `semantic.py` reads `config.registry` → parses YAML → builds two dicts keyed by `name` (`metrics`, `dimensions`) plus the `grains` map (logical → physical relation).
2. **Compile-time integrity** (same checks as `validate` in DEPENDENCY_MAP / Bible §14.5; CI gate A5.2): every `numerator` / `denominator` / `expr` `{token}` resolves to a metric; every metric's `dimensions[]` entry is a defined dimension; every `grain` is in `grains`. Any dangling reference → the server refuses to start (fail loud, never serve a half-valid registry).
3. **Resolve.** `build_query` composes SQL from `sql` / `numerator` / `denominator` / `expr` templates + dimension `sql_expr` + `grains[grain]` only. The model contributes only string names. (Resolver: Bible §14.4; worked example §14.7.)
4. **Govern.** Physical column names live exclusively in registry `sql` fields. A GA4 schema change is a one-file edit; prompts never change.

---

8. Cross-cutting concerns

- **Observability / audit.** The agent-framework control plane (A8.0) wraps every tool call and appends an `audit_log` row (`run_id, step_seq, agent, mcp_tool, args_hash, sql_text, bytes_scanned, latency_ms, verdict, ts`, §22.5). The servers themselves stay thin; the

wrapper is the single instrumentation point — this is what proves "100% governed SQL" (every `sql_text` traces to `semantic-mcp`) and enforces the running byte budget.

- **Idempotency.** stdio servers are pure functions of their inputs (stats seeded). Re-running a window yields byte-identical tool outputs; `save_diagnosis` keys on `finding_id` so re-runs upsert, not duplicate.
- **Versioning.** Each tool manifest carries a `server_version`; `semantic-mcp` also exposes the registry `registry_version` so a finding records the definition version it was built against (Critic checks it, §14.5).
- **Secrets.** Only `warehouse-mcp` and `report-mcp` touch credentials; tokens come from env, never config files. stdio servers run credential-free.

---

9. Reference skeletons

warehouse-mcp (the load-bearing gate — Bible §18.12)

```python
# helios/mcp/warehouse.py
import hashlib
from mcp.server.fastmcp import FastMCP
from google.cloud import bigquery

mcp = FastMCP("warehouse-mcp")
_bq = bigquery.Client()                 # ADC; least-privilege SA
_BYTE_BUDGET = 5_368_709_120            # 5 GiB fixed per-run cap
_SEEN_DRYRUN: set[str] = set()

def _h(sql: str) -> str:
    return hashlib.sha256(" ".join(sql.split()).lower().encode()).hexdigest()

@mcp.tool()
def dry_run(sql: str) -> dict:
    cfg = bigquery.QueryJobConfig(dry_run=True, use_query_cache=False)
    job = _bq.query(sql, job_config=cfg)
    _SEEN_DRYRUN.add(_h(sql))
    b = job.total_bytes_processed
    return {"valid": True, "total_bytes_processed": b,
            "estimated_cost_usd": round(b / 1e12 * 6.25, 5),
            "referenced_tables": [t.table_id for t in job.referenced_tables]}

@mcp.tool()
def run_query(sql: str, max_bytes_billed: int) -> dict:
    if _h(sql) not in _SEEN_DRYRUN:
        raise ValueError("NotDryRunFirst: call dry_run before run_query")   # I1
    cfg = bigquery.QueryJobConfig(maximum_bytes_billed=min(max_bytes_billed, _BYTE_BUDGET))  # I2
    job = _bq.query(sql, job_config=cfg)        # ByteBudgetExceeded → raises
    rows = [dict(r) for r in job.result()]
    return {"rows": rows, "row_count": len(rows),
            "bytes_processed": job.total_bytes_processed, "job_id": job.job_id}

if __name__ == "__main__":
    mcp.run(transport="streamable-http")
```

stats-mcp.decompose_change (the thesis centerpiece — must pass the golden test in §11)

```python
@mcp.tool()
def decompose_change(metric: str, dim: str, t0: list[dict], t1: list[dict]) -> dict:
    s0 = {x["seg"]: x for x in t0}; s1 = {x["seg"]: x for x in t1}
    if set(s0) ^ set(s1):
        raise ValueError("SegmentMismatch: t0/t1 segment sets differ")
    mix = rate = inter = 0.0; by = []
    for s in s0:
        w0, r0, w1, r1 = s0[s]["w"], s0[s]["r"], s1[s]["w"], s1[s]["r"]
        dm, dr, di = (w1-w0)*r0, w0*(r1-r0), (w1-w0)*(r1-r0)   # mix, rate, interaction
        mix += dm; rate += dr; inter += di
        by.append({"seg": s, "mix": dm, "rate": dr, "interaction": di})
    dR = sum(s1[s]["w"]*s1[s]["r"] - s0[s]["w"]*s0[s]["r"] for s in s0)
    return {"delta_R": dR, "mix_effect": mix, "rate_effect": rate,
            "interaction": inter, "by_segment": by}
```

semantic-mcp.build_query (resolver shape — Bible §14.4)

```python
AGG = {"count_distinct": "COUNT(DISTINCT {})", "sum": "SUM({})", "countif": "COUNTIF({})"}
```

```
def _measure(m):  # count/sum/ratio/derived → a SELECT expression
    if m["type"] in ("count", "sum"):   return f"{AGG[m['agg']].format(m['sql'])} AS {m['name']}"
    if m["type"] == "ratio":
        n, d = REG.metrics[m["numerator"]], REG.metrics[m["denominator"]]
        return (f"SAFE_DIVIDE({AGG[n['agg']].format(n['sql'])}, "
                f"{AGG[d['agg']].format(d['sql'])}) AS {m['name']}")
    if m["type"] == "derived":          return f"{expand_expr(m['expr'], REG)} AS {m['name']}"

def build_query(metric, dims, filters, window) -> dict:
    names = metric if isinstance(metric, list) else [metric]
    ms = [REG.metrics.get(n) or _raise("UnknownMetric", n) for n in names]
    grain = ms[0]["grain"]
    for m in ms:
        for d in dims:
            if d not in m["dimensions"]: _raise("DimensionNotPermitted", f"{d} for {m['name']}")
    dim_sql = [f'{REG.dimensions[d]["sql"]} AS {d}' for d in dims]   # UnknownDimension if missing
    where = compile_window(window) + compile_filters(filters, REG)
    grp = f"GROUP BY {', '.join(str(i+1) for i in range(len(dims)))}" if dims else ""
    sql = f"SELECT {', '.join(dim_sql + [_measure(m) for m in ms])} FROM {REG.grains[grain]} WHERE {where} {grp}"
    return {"sql": sql, "governed": True, "metrics": names, "dims": dims}
```

(For `countif(reached_*)` measures, the resolver emits the session-keyed subquery form shown in §6.2 so distinct-session counts and reach flags compose correctly; see Bible §18.5.)

---

10. Per-agent tool allow-lists (AUTHORITATIVE — supersedes Bible §18.9/§19.1)

Servers register globally; each agent sees only these tools (least privilege at the agent layer). **Rule:** every agent with `warehouse-mcp.run_query` also has `dry_run` (G3). The Narrator cannot query; the Critic can re-query to refute.

Agent	Model	Allowed tools
Orchestrator	Opus	`warehouse.list_tables`, `warehouse.describe_table`, `semantic.list_dimensions`, `report.recall_prior`
Monitor	Sonnet	`semantic.get_metric`, `semantic.build_query`, `warehouse.dry_run`, `warehouse.run_query`, `stats.detect_anomaly`, `stats.forecast`
Decompose	Sonnet	`semantic.build_query`, `warehouse.dry_run`, `warehouse.run_query`, `stats.decompose_change`
Diagnose	Opus	`semantic.get_metric`, `semantic.build_query`, `semantic.list_dimensions`, `warehouse.dry_run`, `warehouse.run_query`, `warehouse.reconcile`, `stats.significance_test`, `stats.decompose_change`, `stats.cohort_retention`
Prescribe	Sonnet	`experiment.power_analysis`, `experiment.runtime_estimate`, `experiment.design_experiment`, `semantic.get_metric`
Critic	Opus	`semantic.build_query`, `warehouse.dry_run`, `warehouse.run_query`, `warehouse.reconcile`, `stats.significance_test`, `stats.decompose_change`, `report.recall_prior`
Narrator	Sonnet	`report.render_brief`, `report.save_diagnosis`, `report.export`, `semantic.get_metric`

No agent holds a raw-SQL tool; the only way to data is `build_query → dry_run → run_query`.

---

11. Testing strategy (per server)

Contract + behavior tests; the keystones get golden-value tests because they fail silently.

- **base/contract (all servers):** every tool validates inputs against its JSON Schema; every error path returns the right typed error code (§5); manifest matches this spec.
- **warehouse-mcp:** (a) `run_query` without prior `dry_run` → `NotDryRunFirst`; (b) `max_bytes_billed` capped at budget, over-scan → `ByteBudgetExceeded`; (c) `reconcile` vs a hand-written control query to ≤0.5%; (d) SA is read-only (a write SQL → error).
- **semantic-mcp:** (a) unknown metric/dimension → hard fail (no SQL emitted); (b) `build_query('session_conversion_rate', ['device_category'],window='last_28d')` matches the Bible §14.7 golden SQL (snapshot); (c) registry with a dangling ref → server refuses to start; (d) `DimensionNotPermitted` for a non-whitelisted dim.
- **stats-mcp:** (a) **golden:** `decompose_change` on the §6.2 example → `mix_effect=-0.0018, rate_effect=0, interaction=0, delta_R=-0.0018` (±1e-9); (b) identity holds: `mix+rate+interaction == delta_R` on random inputs; (c) `significance_test` matches a scipy reference; (d) **determinism:** same seed → byte-identical outputs.

- **experiment-mcp:** `power_analysis` two-proportion sample size matches a closed-form reference; `design_experiment` rejects ungoverned metrics.
- **report-mcp:** `render_brief([])` → `EmptyFindings`; `save_diagnosis` → `recall_prior` round-trips (BigQuery + vector ANN); brief faithfulness rule check (every number has a backing tool-output hash).
- **hallucination AST check (shared with eval scorer):** parse every emitted `sql_text`; any column/table not in the registry or GA4 schema → fail. Hard-zero CI gate.

---

12. Build order within the MCP layer (DEPENDENCY_MAP M6/M6b)

1. **A6.0** `base.py` — server factory, error types, schemas, audit wrapper. (Unblocks all servers.)
2. **A6.2** `semantic-mcp` — needs the A5.1 registry (already built ✓). Build first of the "real" servers; everything queryable depends on it.
3. **A6.1** `warehouse-mcp` — pair with semantic-mcp; verify the `build_query → dry_run → run_query → reconcile` round-trip end-to-end (M6 gate).
4. **A6.3** `stats-mcp` ‖ — data-independent; build in parallel from day one with the `decompose_change` golden test (M6b).
5. **A6.4** `experiment-mcp` ‖ — data-independent (M6b).
6. **A6.5** `report-mcp` **core** (`render_brief` / `export`) — stateless; needed for the L1 minimal loop (M6b/M7).
7. **A7.4** `report-mcp` **memory tools** — after the `helios_memory` DDL + vector store land (M8).

M6 exit gate: an agent can name a metric → governed SQL is built, dry-run-costed, executed under budget, reconciled to ≤0.5%, and decomposed — with zero hand-authored SQL anywhere in the path.